

Tartalomjegyzék

GPT 4.1 promptjai.....	2
1. Felhasználói kérdés.....	2
Válasz.....	2
2. Felhasználói kérdés.....	2
Válasz.....	2
3. Felhasználói kérdés.....	2
Válasz.....	2
4. Felhasználói kérdés.....	2
Válasz.....	2
5. Felhasználói kérdés.....	2
Válasz.....	3
6. Felhasználói kérdés.....	3
Válasz.....	3
7. Felhasználói kérdés.....	3
Válasz.....	3
8. Felhasználói kérdés.....	3
Válasz.....	3
9. Felhasználói kérdés.....	3
Válasz.....	3
10. Felhasználói kérdés.....	3
Válasz.....	3
11. Felhasználói kérdés.....	3
Válasz.....	4
12. Felhasználói kérdés.....	4
Válasz.....	4
13. Felhasználói kérdés.....	4
Válasz.....	4
14. Felhasználói kérdés.....	4
Válasz.....	4
15. Felhasználói kérdés.....	4
Válasz.....	4
16. Felhasználói kérdés.....	4
Válasz.....	5
17. Felhasználói kérdés.....	5
Válasz.....	5
18. Felhasználói kérdés.....	5
Válasz.....	5
Sikeres lépések.....	5
Nem sikeres vagy még javítandó részek.....	6
Összefoglaló.....	6
Codex promptjai.....	6
Codex prompt értékelések indoklások.....	23

GPT 4.1 promptjai

1. Felhasználói kérdés

• Szia AI! Ugye te azt teljesen jó érted, ha azt kérdezem tőled, hogy én akarok egy bevásárló lista alkalmazást, tudod, hogy miről beszélek?

Válasz

• Igen, teljesen értem! Egy bevásárló lista alkalmazás általában egy egyszerű app, ahol a felhasználók listákat hozhatnak létre, elemeket adhatnak hozzá (pl. termékek neve, mennyisége), jelölhetik őket késznek, törölhetik őket, és esetleg kategorizálhatják vagy menthetik a listákat.

2. Felhasználói kérdés

• Szeretnék egy MEAN stack projektet készíteni. Írd le a projekt fő komponenseit, a mappa-struktúrát, és javasolj egy minimális működőképes verziót (MVP). A projekt témája: Egy bevásárló lista alkalmazás...

Válasz

- Fő komponensek: Backend (Node.js + Express), MongoDB, Angular frontend, CORS, környezeti változók.
- Mappa struktúra: backend és frontend könyvtárak.
- MVP: lista létrehozása, elem hozzáadása, kész jelölése, törlés, listanézet.

3. Felhasználói kérdés

• Hogyan érdemes veled egy ilyen projektet leimplementálni, hogy ne legyen megakadás? ...

Válasz

- Iteratív, moduláris haladás: alapok, teszt, majd bővítés.
- Konkrét lépések: backend, API végpont, frontend komponens, integráció.

4. Felhasználói kérdés

• Implementáld az alap backend szerveret!

Válasz

- Backend könyvtár létrehozása.
- package.json, server.js, .env létrehozása Express + Mongoose + CORS.
- npm install futtatása.

5. Felhasználói kérdés

• Mi a hiba oka?

Válasz

- Az `npm start` parancsot a gyökérben futtattad, nem a backend mappában. A package.json a backend alatt van.

6. Felhasználói kérdés

- A `mongodb - t` dockeresen állítsd össze, dokker fileból legyen indítható.

Válasz

- `docker-compose.yml` létrehozása MongoDB szolgáltatással.
- `.env` frissítése Docker Mongo URI-val.
- Docker telepítés ellenőrzése és konténer indítása.

7. Felhasználói kérdés

- A `warning` mire vonatkozik?

Válasz

- A `docker-compose.yml` `version: '3.8'` sora elavult, figyelmen kívül hagyja.

8. Felhasználói kérdés

- A `git init` parancsot itt a gyökér mappában kell kiadni?

Válasz

- Igen, a projekt gyökerében kell. Meg is történt.

9. Felhasználói kérdés

- Eddig melyik az a fájl vagy mappa amelyiket nem érdemes, nem biztonságos feltenni a github -ra?

Válasz

- `.env`, `node_modules/`, logok, IDE mappák.
- `.gitignore` létrehozása.

10. Felhasználói kérdés

- A githubon létre van hozva az üres repo, milyen parancsokkal tudom a projektet feltölteni?

Válasz

- `git add .`
- `git commit -m "..."`
- `git remote add origin ...`
- `git push -u origin master`

11. Felhasználói kérdés

- Kérlek javasolj api végpontokat az eddig specifikált funkciókhoz!

Válasz

- GET /api/lists
- POST /api/lists
- GET /api/lists/:id
- PUT /api/lists/:id
- DELETE /api/lists/:id
- GET /api/lists/:listId/items
- POST /api/lists/:listId/items
- PUT /api/lists/:listId/items/:itemId
- DELETE /api/lists/:listId/items/:itemId

12. Felhasználói kérdés

- Igen, ezeket mind implementáld!

Válasz

- Modellek és route-ok létrehozása.
- List.js, Item.js, lists.js implementáció.
- Backend frissítése route regisztrációval.

13. Felhasználói kérdés

- Kérlek csinálj mindegyik végponthoz néhány teszt esetet és tedd őket egy fájlba...

Válasz

- test.js létrehozása backendben.
- npm test hozzáadása package.json-hoz.
- E2E tesztek listákhoz és elemekhez.

14. Felhasználói kérdés

- a user és az admin létrehozását mikor javaslod?

Válasz

- Frontend után, de közvetlenül az integráció előtt.
- Előbb MVP, aztán autentikáció.

15. Felhasználói kérdés

- Ha szeretnék egy olyan funkciót ... hogy a user lista itemjeit sorba is tudja rakni ...

Válasz

- Most érdemes backendben figyelni a sorrendre.
- order mező és PUT /api/lists/:listId/reorder.

16. Felhasználói kérdés

- Módosítsd az Item modellt order mezővel, és készítsd el a reorder API végpontot

Válasz

- Item modell módosítása order mezővel.
- `reorder` végpont hozzáadása.
- Tesztfrissítés.

17. Felhasználói kérdés

- `mi a curl hívása a sorrend módosításhoz?`

Válasz

```
curl -X PUT http://localhost:3000/api/lists/{listId}/reorder -H "Content-Type: application/json" -d '{"itemIds": ["itemId1", "itemId2"]}'
```

18. Felhasználói kérdés

- `hogyan szerezem meg a <PID> -et?`

Válasz

- `lsof -i :3000`
- `ps aux | grep node`

Sikeres lépések

1.Backend alapok létrehozása

- package.json, server.js, .env
- Miért sikeres: a kód elkészült, a szerver elindult a backend mappában.

2.Docker Compose MongoDB beállítása

- docker-compose.yml létrehozása MongoDB-vel
- `.env` frissítése Dockeres URI-val
- Miért sikeres: a MongoDB konténer elindult, a backend csatlakozni tudott hozzá.

3.Git repository inicializálása és feltöltése

- .gitignore készítése
- `git init`, `git add .`, `git commit`, `git push`
- Miért sikeres: a projekt felkerült GitHub-ra és a változások verziókezelve vannak.

4.API végpontok implementálása

- List és Item modellek
- `GET/POST/PUT/DELETE` listákhoz
- `GET/POST/PUT/DELETE` elemekhez
- Miért sikeres: a backend API-ok készen lettek és használhatók.

5.Tesztfájl létrehozása

- test.js
- `npm test` script hozzáadása

- Miért sikeres: egy parancsra futtatható tesztfájl áll rendelkezésre.

6.Sorrend mentés funkció hozzáadása

- order mező az Item modellben
- `PUT /api/lists/:listId/reorder`

- Miért sikeres: a backend készen áll a sorrend mentésére, és a GET lekérés rendezetten adja vissza az elemeket.
-

Nem sikeres vagy még javítandó részek

1.`npm start` rossz mappában

- Hiba: package.json nem volt a gyökérben
- Miért nem sikeres: a parancs a projekt gyökérben futott, nem a backend mappában.

2.Docker permission issue

- Hiba: `permission denied while trying to connect to the docker API`
- Miért nem sikeres: Docker sockethoz nem volt jogosultság, sudo kellett.

3.`reorder` végpont kezdeti 404-es hiba

- Hiba: a routeok sorrendje miatt az Express rosszul értelmezte a `reorder`-t
- Miért nem sikeres: a specifikus útvonalakat korábban kellett volna elhelyezni.

4.Port ütközés a 3000-es porton

- Hiba: `EADDRINUSE: address already in use :::3000`
- Miért nem sikeres: volt már futó szerver a 3000-es porton, ezért nem indult új példány.

5.`npm test` `ECONNREFUSED`

- Hiba: a teszt a backendhez próbált kapcsolódni, de a szerver nem futott
 - Miért nem sikeres: maga a teszt logikája jó, de a futtatási környezet nem volt elindítva.
-

Összefoglaló

- A backend implementáció és a funkcionalitások nagyrészt késznek tekinthetők.
- A fő problémák inkább környezeti/első indítási hibák voltak, nem a logikai implementáció hibái.
- Ha szeretnéd, segítek most abban, hogy a szerver stabilan fusson, és utána mehet a frontend.

Codex promptjai

PROMPT:

látod a szessönt?

VÁLASZ:

Igen, látom a session kontextusát: a workspace a Rf_Dinnyes_balazs01 mappában van, az aktív fájl a backend/src/routes/lists.js, és látom a megnyitott tabokat is: Item.js, List.js, .gitignore, server.js. Nem látom automatikusan a fájlok teljes tartalmát, de be tudom olvasni őket, ha ránézzek valamire.

PROMPT:

Most kérlek adj rövid útmutatást, lépéseket a frontend és a többi lépés létrehozására. Milyen promptokat vársz? Amint látod, egy bevásárló lista rendszert fejleszték.

VÁLASZ:

Egy bevásárlólista rendszerhez a javasolt lépések:

1. Backend stabilizálása.
2. Route-ok ellenőrzése listákhoz és itemekhez.
3. Angular vagy React frontend létrehozása.
4. API kapcsolat kialakítása.
5. Alap funkciók megvalósítása: lista létrehozás, item hozzáadás, kipipálás, törlés.
6. Később login, admin, kategóriák, boltok, keresés, rendezés.

Javasolt promptok:

- Nézd át a backend jelenlegi route-jait és modelleket.
- Egészítsd ki a backend API-t teljes CRUD működéssel.
- Hozz létre egy React/Vite vagy Angular frontendet.
- Kösd össze a frontendet a backend /lists route-jaival.
- Készíts egy modern bevásárlólista felületet.
- Futtasd le az appot és javítsd ki a hibákat.

PROMPT:

Én most mean stack - ben dolgozok. Hozd létre az alap angular projektet!

VÁLASZ:

Létrejött az Angular alap frontend a frontend/ mappában.

Beállítások:

- Angular 19
- routing
- SCSS
- HttpClient provider
- backend proxy: /api -> http://localhost:3000
- alap kezdőképernyő
- modellek és service a bevásárlólista API-hoz

Fontos fájlok:

- frontend/src/app/models/shopping-list.model.ts
- frontend/src/app/services/lists.service.ts
- frontend/proxy.conf.json

A frontend dev szerver:

http://localhost:4200/

Ellenőrzés:

npm run build sikeresen lefutott.

PROMPT:

miután létrehoztad az angular frontendet a mostani backendhez, kösd össze a frontendet a backend /lists route-jaival.

VÁLASZ:

Az Angular frontend össze lett kötve a backend /api/lists route-jaival.

Megvalósított funkciók:

- GET /api/lists
- POST /api/lists
- DELETE /api/lists/:id
- GET /api/lists/:listId/items
- POST /api/lists/:listId/items
- PUT /api/lists/:listId/items/:itemId
- DELETE /api/lists/:listId/items/:itemId

Fontos fájlok:

- frontend/src/app/app.component.ts
- frontend/src/app/app.component.html
- frontend/src/app/services/lists.service.ts
- frontend/proxy.conf.json

A frontend fut:

<http://localhost:4200/>

PROMPT:

Készíts egy egyszerű, modern bevásárlólista felületet: bal oldalt listák, jobb oldalt a kiválasztott lista elemei.

VÁLASZ:

Elkészült egy modern kétpaneles bevásárlólista felület.

Bal oldalt:

- listák megjelenítése
- új lista létrehozása
- aktív lista kijelölése

Jobb oldalt:

- kiválasztott lista fejléc
- összes elem / kész elemek / becsült összeg
- új termék hozzáadása névvel, mennyiséggel, árral
- termék kipipálása
- termék törlése
- lista törlése

Módosított fájlok:

- frontend/src/app/app.component.html
- frontend/src/app/app.component.scss

Ellenőrzés:

npm run build sikeresen lefutott.

PROMPT:

Most kérek azt valósítsd meg, hogy látsz a backendnél egy item sorba rendező funkciót, kérlek azt vidd át a frontendre, hogy a listák itemjeit moussal lehessen sorba rendezni húzással!

VÁLASZ:

Megvalósult az itemek húzással történő rendezése.

Működés:

- az item sor elején megjelent egy :: fogantyú
- egérrel húzható az item
- drop után a frontend meghívja:
PUT /api/lists/:listId/reorder
- a sorrend azonnal frissül a UI-ban
- backend menti az új sorrendet
- hiba esetén visszaáll az előző sorrend

Módosított fájlok:

- frontend/src/app/app.component.ts
- frontend/src/app/app.component.html
- frontend/src/app/app.component.scss

PROMPT:

Most kellene egy admin, user autentikáció megvalósítása. Az adminnak kellene egy külön menedzser oldal, ahol törölheti a usereket, ha akarja, meg látja a regisztrált adatait. A usereknek is hozzd létre a regisztráló, bejelentkező felületet. A jelszó legyen hash -elve mentve és kezelve megfelelően az adattbázisban!

VÁLASZ:

Megvalósult az admin/user autentikáció.

Backend:

- új User modell
- bcryptjs jelszó hash
- JWT alapú auth
- auth middleware
- admin route-ok

Új backend fájlok:

- backend/src/models/User.js
- backend/src/routes/auth.js
- backend/src/routes/admin.js
- backend/src/middleware/auth.js

Route-ok:

- POST /api/auth/register
- POST /api/auth/login

- GET /api/auth/me
- GET /api/admin/users
- DELETE /api/admin/users/:id

Frontend:

- login felület
- regisztrációs felület
- token mentése localStorage-ba
- Authorization Bearer token automatikus küldése
- kijelentkezés
- admin panel
- userek listázása
- user törlése

Fontos:

Az első regisztrált felhasználó automatikusan admin lesz.

PROMPT:

gondolom az adminnak valami előre létrehozott bejelentkező adatai vannak, azt oszd meg itt velem.

VÁLASZ:

Nincs előre létrehozott admin belépési adat.

Az első regisztrált felhasználó automatikusan admin lesz.

Példa első regisztráció:

Név: Admin

Email: admin@example.com

Jelszó: admin123

Ha már van user az adatbázisban, akkor az új regisztráció már sima user lesz.

PROMPT:

azt írja, hogy sikertelen regisztráció, nem enged semmilyen adattal regisztrálni

VÁLASZ:

A hiba oka az volt, hogy a 3000-es porton egy régi backend példány futott, amelyben még nem volt /api/auth/register route.

Megoldás:

- régi backend folyamatok leállítása
- aktuális auth-os backend újraindítása

Teszt eredmény:

POST /api/auth/register -> 201 Created

A teszt közben létrejött admin user:

Email: testuser@example.com

Jelszó: admin123

Role: admin

A frontend auth hibaüzenet is javítva lett, hogy a backend konkrét hibáját jelenítse meg.

PROMPT:

ha egy listát törölni akarok, akkor kérdezzen rá hogy tényleg törölni akajra - e a user vagy admin. A jelszó mezőnél, mind a bejelentkezésnél, mind a regisztrációnl legyen láthatósági opció, illetve a regisztrációnál legyen dupla egyezőség ellenőrzési beviteli lehetőség!

VÁLASZ:

Megvalósult:

- lista törlés előtt megerősítő ablak
- admin user törlés előtt megerősítő ablak
- login jelszómezőnél Mutat / Elrejt gomb
- regisztrációnál jelszó láthatóság opció
- regisztrációnál jelszó megerősítő mező
- a két jelszónak egyeznie kell, különben nem engedi elküldeni

Módosított fájlok:

- frontend/src/app/app.component.ts
- frontend/src/app/app.component.html
- frontend/src/app/app.component.scss

PROMPT:

Az oldalakra mindenhol tegyél egy halovány kosár vagy valami étel vagy gyümölcs hátteret, mindenhol ugyanaz legyen.

VÁLASZ:

Minden oldal egységes halvány bevásárlós/gyümölcsös hátteret kapott.

Új asset:

- frontend/public/food-pattern.svg

Megjelenik:

- bejelentkezésnél
- regisztrációnál
- főoldalon
- admin felületen

Módosított fájl:

- frontend/src/app/app.component.scss

PROMPT:

A főoldalon lehessen váltani sötét vagy világos megjelentítés között egy sarki ikonnal, ami egy szép ikon legyen

VÁLASZ:

Megvalósult a világos/sötét mód váltás.

Működés:

- jobb felső sarokban ikon gomb
- világos módban hold ikon látszik
- sötét módban nap ikon látszik
- kattintásra vált a téma
- választás localStorage-ba mentődik
- frissítés után is megmarad

A sötét mód érinti:

- háttér
- paneleket
- listákat
- inputokat
- admin sort
- gombokat

PROMPT:

ha az admi törölni akar egy usert, akkor kérdezzen rá egy ablak, illetve a törlés sikerességét egy felugró snackbar jelezze 2 másodpercig.

VÁLASZ:

Admin user törlésnél:

- először megerősítő ablak jelenik meg
- jóváhagyás és sikeres törlés után snackbar jelenik meg
- snackbar szövege:
Felhasználó sikeresen törölve.
- 2 másodperc után automatikusan eltűnik
- világos és sötét módban is működik

PROMPT:

Kellene még, hogy a user tudjon keresni a listái között, kell egy kereső mező és ahogy írom be a nevet változzon a listák listája is.!

VÁLASZ:

Megvalósult a listák közötti keresés.

Funkciók:

- keresőmező a bal oldali listapanelen
- gépelés közben azonnali szűrés
- lista név alapján keres
- ha nincs találat, megjelenik:

Nincs találat

Módosított fájlok:

- frontend/src/app/app.component.ts
- frontend/src/app/app.component.html
- frontend/src/app/app.component.scss

PROMPT:

most jelen pillanatban hány adatbázis entitást/kollekciót a használ a projekt?

VÁLASZ:

A projekt ekkor 3 adatbázis entitást / MongoDB kollekciót használt:

1. User

Kollekció: users

Felhasználók, admin/user role, email, hash-elt jelszó

2. List

Kollekció: lists

Bevásárlólisták, userhez kötve userId alapján

3. Item

Kollekció: items

Listaelemek, listához kötve listId alapján

Kapcsolatok:

User 1 - N List

List 1 - N Item

PROMPT:

mMilyen további 2 kollekcióval lehetne bővíteni az adatbázist ebben a bevásárló lista alkalmazásban? Szerinted?

VÁLASZ:

Két javasolt új kollekció:

1. Category / Kategória

Kollekció: categories

Példák:

- Zöldség
- Gyümölcs
- Tejtermék
- Hús
- Pékáru
- Háztartás

Haszna:

- rendezettebb bevásárlólista
- szűrés kategória szerint
- statisztika kategóriánként
- színek/ikonok később

Kapcsolat:

Category 1 - N Item

Item -> categoryId

2. Store / Bolt

Kollekció: stores

Példák:

- Lidl
- Aldi
- Tesco
- Spar
- Piac
- DM

Haszna:

- bevásárlólisták boltonként
- itemek csoportosítása bolt szerint
- kedvenc bolt mentése userenként

Kapcsolat:

Store 1 - N List

List -> storeId

PROMPT:

Ok ezeket valósítsd meg ilyen sorrendben. Árak összehasonlítása nem kell.

VÁLASZ:

Megvalósult két új kollekció:

1. Category / Kategória

Backend model:

- backend/src/models/Category.js

Backend route:

- backend/src/routes/categories.js

API:

- GET /api/categories
- POST /api/categories
- DELETE /api/categories/:id

Frontend:

- új kategória létrehozható
- itemhez kategória rendelhető
- item sorban megjelenik a kategória neve

2. Store / Bolt

Backend model:

- backend/src/models/Store.js

Backend route:

- backend/src/routes/stores.js

API:

- GET /api/stores
- POST /api/stores
- DELETE /api/stores/:id

Frontend:

- új bolt létrehozható
- aktuális listához bolt rendelhető
- bolt neve megjelenik a lista neve alatt

Az ár-összehasonlítás nem készült el, mert nem volt kérve.

Új kollekciók:

- users
- lists
- items
- categories
- stores

PROMPT:

most állítsd le és indítsd újra a mongot

VÁLASZ:

A MongoDB Docker konténert nem lehetett innen újraindítani, mert a Docker sockethez nem volt jogosultság.

Hiba:

permission denied while trying to connect to the docker API at unix:///var/run/docker.sock

Javasolt parancs:

```
cd /home/bakisz/Gi_msc_szoftveres_targyai_/4th_semester/Rf_Dinnyes_balazs01
```

```
sudo docker compose restart mongodb
```

Ellenőrzés:

```
sudo docker compose ps
```

PROMPT:

le kellen újra állítani és indítani a mongot

VÁLASZ:

A MongoDB Docker konténert nem lehetett innen újraindítani, mert a Docker sockethez nem volt jogosultság.

Javasolt parancs:

```
cd /home/bakisz/Gi_msc_szoftveres_targyai_/4th_semester/Rf_Dinnyes_balazs01
```

```
sudo docker compose restart mongodb
```

Ellenőrzés:

```
sudo docker compose ps
```

Ha a backend nem csatlakozik vissza:

```
cd backend
```

npm run dev

PROMPT:

Error: listen EADDRINUSE: address already in use :::3000

VÁLASZ:

A hiba oka az volt, hogy több backend / nodemon példány futott egyszerre, és foglalták a 3000-es portot.

Megoldás:

- duplikált backend folyamatok megkeresése
- régi node/nodemon példányok leállítása
- backend újraindítása egy példányban

Eredmény:

Server running on port 3000

MongoDB connected

PROMPT:

Mikor a boltot kiválasztom, akkor a lista nevénél is változzon aktualizálódjon!

VÁLASZ:

Megvalósult, hogy az aktuálisan kiválasztott listánál a bolt módosítható legyen.

Működés:

- a kiválasztott lista neve alatt megjelent egy Bolt választó
- bolt kiválasztásakor a frontend elküldi a módosítást a backendnek
- a kiválasztott lista frissül

- a bal oldali listanézetben a lista neve alatti bolt is azonnal aktualizálódik
- siker esetén snackbar jelenik meg:
A lista boltja frissítve.

Módosított fájlok:

- frontend/src/app/app.component.ts
- frontend/src/app/app.component.html
- frontend/src/app/app.component.scss

PROMPT:

a bal oldali sávból kiveheted a boltválasztót, nincs funkciója

VÁLASZ:

A bal oldali sávból ki lett véve az új lista létrehozásánál lévő boltválasztó.

Most:

- bal oldalt csak lista neve alapján lehet listát létrehozni
- boltot továbbra is az aktuális kiválasztott lista fejlécében lehet hozzárendelni/módosítani
- a felesleges newListStoreId frontend állapot is ki lett szedve

Ellenőrzés:

npm run build sikeresen lefutott.

Codex prompt értékelések indoklások

PROMPT ÉRTÉKELÉSI ÖSSZEFOGLALÓ

1. Prompt: látod a szessönt?

Állapot: Sikeres

Indoklás: A session kontextus, workspace és nyitott fájlok azonosítva lettek.

2. Prompt: frontend és további lépések útmutatója

Állapot: Sikeres

Indoklás: Készült rövid fejlesztési terv és javasolt promptlista.

3. Prompt: Angular alap projekt létrehozása MEAN stackhez

Állapot: Sikeres

Indoklás: Létrejött a frontend Angular projekt routinggal, SCSS-sel, proxyval és alap service/model réteggel.

4. Prompt: frontend összekötése a backend /lists route-jaival

Állapot: Sikeres

Indoklás: A frontend meghívja a list és item CRUD végpontokat.

5. Prompt: modern kétpaneles bevásárlólista felület

Állapot: Sikeres

Indoklás: Elkészült a bal oldali listapanel és jobb oldali item panel.

6. Prompt: itemek húzással rendezése

Állapot: Sikeres

Indoklás: A frontend drag and drop rendezést kapott, és menti a sorrendet a backend reorder route-on.

7. Prompt: admin/user autentikáció, hash-elt jelszóval

Állapot: Sikeres

Indoklás: Elkészült a User modell, bcrypt hash, JWT auth, login/register felület és admin user manager.

8. Prompt: admin belépési adatok megosztása

Állapot: Részben sikeres

Indoklás: Nem volt előre létrehozott admin adat, de tisztázva lett, hogy az első regisztrált user lesz admin.

9. Prompt: sikertelen regisztráció javítása

Állapot: Sikeres

Indoklás: Kiderült, hogy régi backend példány futott. Újraindítás után a regisztráció működött.

10. Prompt: lista törlés megerősítés, jelszó láthatóság, dupla jelszó regisztrációnál

Állapot: Sikeres

Indoklás: Minden kért UI és validációs funkció megvalósult.

11. Prompt: halvány bevásárlós/ételes/gyümölcsös háttér minden oldalon

Állapot: Sikeres

Indoklás: Készült közös food-pattern.svg háttér, amely minden fő nézeten megjelenik.

12. Prompt: világos/sötét mód sarki ikonnal

Állapot: Sikeres

Indoklás: Elkészült a jobb felső ikon alapú téma váltó localStorage mentéssel.

13. Prompt: admin user törlés megerősítés és snackbar

Állapot: Sikeres

Indoklás: Megmaradt a confirm ablak, és sikeres törlés után 2 másodperces snackbar jelenik meg.

14. Prompt: user tudjon keresni a listái között

Állapot: Sikeres

Indoklás: Elkészült a bal oldali keresőmező, amely gépelés közben szűri a listákat.

15. Prompt: hány adatbázis entitást használ a projekt

Állapot: Sikeres

Indoklás: Az akkori állapot alapján helyesen azonosítva lett a 3 kollekció: users, lists, items.

16. Prompt: javasolj további 2 kollekciót

Állapot: Sikeres

Indoklás: Javaslat született Category és Store kollekciókra, rövid indoklással.

17. Prompt: Category és Store megvalósítása, ár-összehasonlítás nélkül

Állapot: Sikeres

Indoklás: Létrejött a categories és stores kollekció, backend route, frontend létrehozás és hozzárendelés.

18. Prompt: MongoDB leállítása és újraindítása

Állapot: Sikertelen

Indoklás: Docker socket jogosultság hiánya miatt nem lehetett végrehajtani innen. Sudo parancs lett megadva manuális futtatáshoz.

19. Prompt: MongoDB újraindítása újra

Állapot: Sikertelen

Indoklás: Ugyanaz a Docker jogosultsági probléma állt fenn.

20. Prompt: EADDRINUSE hiba javítása

Állapot: Sikeres

Indoklás: Több backend/nodemon folyamat futott. A duplikált folyamatok le lettek állítva, majd a backend újraindult.

21. Prompt: bolt kiválasztásakor a lista neve melletti adat frissüljön

Állapot: Sikeres

Indoklás: Az aktuális lista fejlécébe bekerült a bolt választó, amely menti és frissíti a bal oldali listanézetet is.

22. Prompt: bal oldali sávból boltválasztó kivétele

Állapot: Sikeres

Indoklás: Az új lista létrehozásából ki lett véve a boltválasztó, a bolt módosítása csak az aktuális lista fejlécében maradt.

23. Prompt: eddigi promptok és válaszok kimásolható összefoglalása

Állapot: Sikeres

Indoklás: Készült másolható összefoglaló a főbb promptokról és válaszokról.

24. Prompt: minden másolható formában legyen

Állapot: Sikeres

Indoklás: Az összefoglaló plain text kó blokkban készült.

25. Prompt: promptok és válaszok összefoglalása egyben másolható formában

Állapot: Sikeres

Indoklás: Készült promptonként bontott, egyben másolható összefoglaló.

ÖSSZESÍTÉS

Sikeres promptok száma: 22

Részben sikeres promptok száma: 1

Sikertelen promptok száma: 2

Sikertelen promptok oka:

- A MongoDB Docker konténer újraindítását a rendszer jogosultsági okból nem engedte.
- A docker API sockethez nem volt hozzáférés.
- A sudo futtatáshoz jelszó kellett, amit az asszisztens nem tud megadni.

Részben sikeres prompt oka:

- Admin belépési adat nem létezett előre, de a rendszer működése tisztázva lett, és létrejött admin user az első regisztrációval.